

COMPUTER ARCHITECTURE



Week 2: Computer Performance

Learning Objectives

01

Define performance and explain why it is always context-dependent

02

Apply the Iron Law to calculate CPU execution time

03

Distinguish latency from throughput and know when each is appropriate

04

Evaluate benchmark results and explain their limitations

05

Explain why clock-rate scaling stalled around 2005 (the Power Wall)

06

Use Amdahl's Law to quantify the limits of any single optimization

Content

1

Introduction

Why performance matters - Architect's trade-off

2

Performance Metrics

Latency (CPU time) - Throughput (MIPS/FLOPS) - Benchmarks

3

The Power Wall

Dennard scaling - CMOS power - Frequency limits

4

Amdahl's Law

Speedup formula - Limits of parallelism

5

Future Trends

DSAs - Compiler co-design - Chiplets - Near-memory computing

01

Introduction

Why performance matters - Architect's trade-off

Why Faster Computers Matter



Economy & Society

Faster computation underpins finance, logistics, communications, and every modern industry.



New Applications

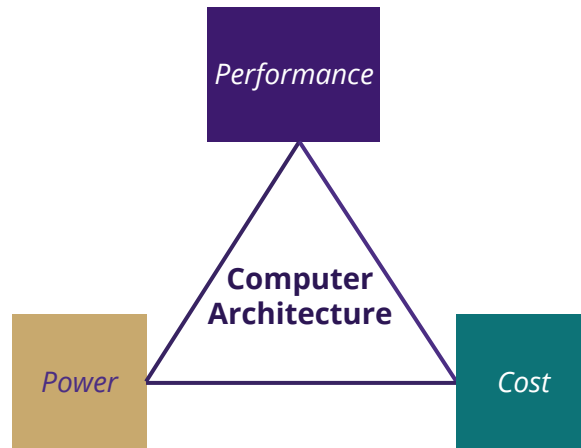
Many capabilities - from real-time AI to genome sequencing - simply did not exist without sufficient compute.



Data Deluge

The volume of data generated globally grows exponentially, demanding ever more powerful machines to process it.

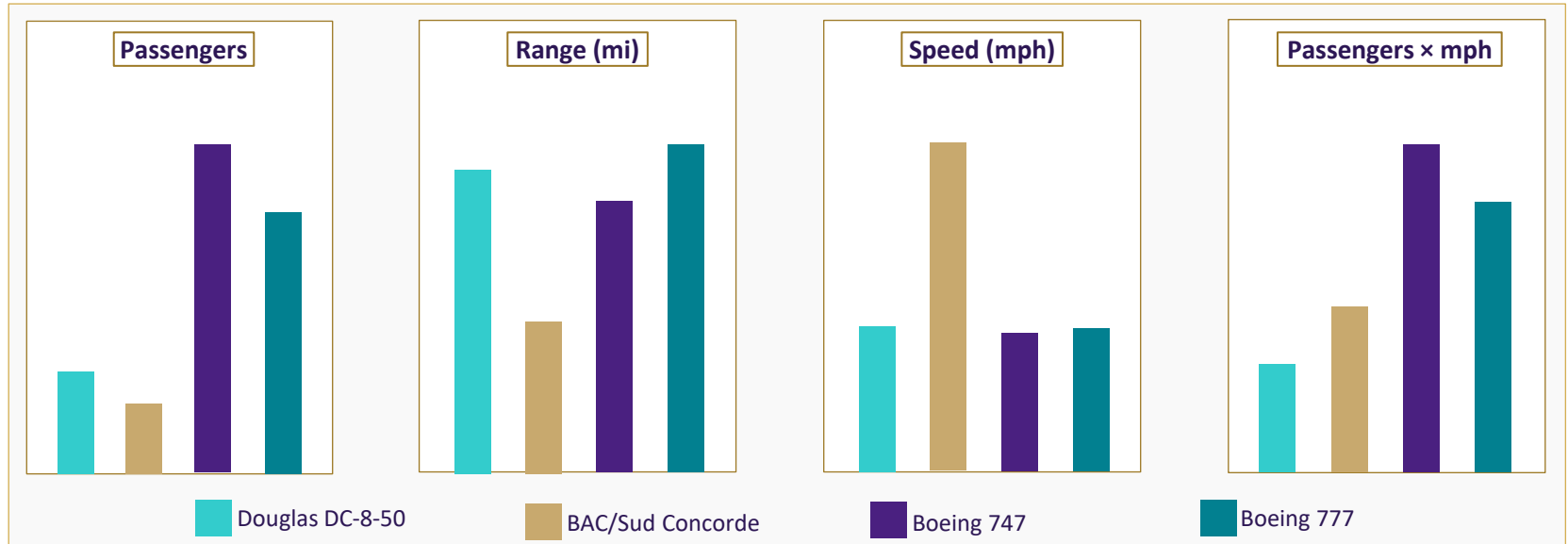
The Architect's Dilemma



Goal: find the optimal balance for a specific design target.

Defining Performance: Depends on Context

Which airplane has the best performance?



Key insight: *Performance is relative. You must define your metric before comparing systems.*

02

Performance Metrics

Latency (CPU time) - Throughput (MIPS/FLOPS) - Benchmarks

Two Core Metrics for Computer Performance



LATENCY

Definition: Time between start and completion of a task.

Also called: Response time, execution time, elapsed time.

Primary metric in this course:

CPU Time (excludes I/O & OS overhead)



THROUGHPUT

Definition: Total work done per unit time.

Examples: transactions/hour, MIPS, GFLOPS.

Use when: evaluating servers, pipelines, or parallel workloads where volume matters more than individual task time.

Performance is relative: $Speedup = \frac{Performance_x}{Performance_y} = \frac{Execution\ time_y}{Execution\ time_x} = n$

Execution Time: Elapsed vs. CPU Time

ELAPSED TIME (Wall-Clock Time)

- > Total response time including I/O waits, OS interrupts, and other processes sharing the CPU.
- > Useful for end-users; too noisy for hardware designers.
- > Measured with the time command in Linux.

CPU TIME ← Our Focus

- > Time the CPU actually spent executing the program. Excludes I/O time and other processes.
- > Based on the CPU's clock — the oscillating signal synchronizing all CPU operations.

$$\text{CPU Time} = \text{\#Clock Cycles} \times \text{Clock Period} = \text{\#Clock Cycles} / \text{Clock Rate}$$

The Iron Law of CPU Performance

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Period} = (\text{IC} \times \text{CPI}) / \text{Clock Rate}$$

Instruction Count (IC)

Number of instructions executed.
Affected by: algorithm, ISA, compiler

CPI (Cycles per Instruction)

Average cycles needed per instruction.
Affected by: CPU design, instruction mix

Clock Rate

Cycles per second (Hz).
Affected by: fabrication technology, μ -architecture

For a mix of instructions, CPI represents the **average** number of cycles per instruction:
$$CPI = \frac{\# \text{Clock cycles}}{IC} = \sum_{i=1}^n \left(CPI_i \times \frac{IC_i}{IC} \right)$$

Software factors: *Algorithm & data structures* → IC, CPI | *Programming language* → IC, CPI | *Compiler quality* → IC, CPI

Iron Law: Example

Given instruction mix:

Instruction Class	Frequency	CPI
ALU	50%	1
Load	20%	5
Store	10%	3
Branch	20%	2

Q1: What is the overall CPI?

$$\begin{aligned}\text{CPI} &= 0.5 \times 1 + 0.2 \times 5 + 0.1 \times 3 + 0.2 \times 2 \\ &= 0.5 + 1.0 + 0.3 + 0.4 = 2.2\end{aligned}$$

Q2: How much faster if Load CPI drops to 2?

$$\begin{aligned}\text{New CPI} &= 0.5 \times 1 + 0.2 \times 2 + 0.1 \times 3 + 0.2 \times 2 = 1.6 \\ \text{Speedup} &= 2.2 / 1.6 = 1.38\times \text{ faster}\end{aligned}$$

Q3: What if 2 ALU instructions execute per cycle?

$$\begin{aligned}\text{New ALU CPI} &= 0.5 \rightarrow \text{New CPI} = 0.25 + 1.0 + 0.3 + 0.4 = 1.95 \\ \text{Speedup} &= 2.2 / 1.95 = 1.13\times \text{ faster}\end{aligned}$$

💡 Reducing Load CPI (20% of instructions) yields bigger gains than doubling ALU throughput (50% of instructions).
Always optimize the bottleneck!

Throughput Metrics: MIPS & FLOPS

$$\text{MIPS} = \text{Instruction Count} / (\text{Execution Time} \times 10^6) = \text{Clock Rate} / (\text{CPI} \times 10^6)$$

✓ Why It's Used

- > Easy to understand and communicate
- > Larger MIPS → often (but not always) faster machine
- > FLOPS variants (GFLOPS, TFLOPS) widely used for scientific/AI workloads

✗ Critical Drawbacks

- > Cannot compare machines with different ISAs
- > Varies between programs on the same machine
- > **Higher MIPS can coexist with worse CPU time!**

Counter-example: Which compiler is better? (4 GHz machine)

Compiler	Class A (CPI=1)	Class B (CPI=2)	Class C (CPI=3)	CPU Time	MIPS
Compiler 1	5B	1B	1B	2.5 s ✓	2800
Compiler 2	10B	1B	1B	3.75 s ✗	3200 ← misleading!

Composite Metrics: Benchmarks

Most real-world users rely on benchmarks to evaluate computer performance.

What Is Benchmarking?

- > Run standardized programs to measure performance
- > Reflects **real** workloads, not only stress tests

What Makes a Good Benchmark?

- > Represents the target application domain (e.g., compilers, gaming, AI)
- > Uses inputs similar to real-world data

SPEC CPU (industry standard)

- > Vendor-agreed programs & inputs
- > Benchmarks for CPU performance
- > Result: SPECratio (geometric mean)

Sample SPEC CPU results:

Benchmark	IC ($\times 10^9$)	CPI	Exec Time (s)	SPECratio
perlbench	2684	0.42	627	2.83
gcc	2322	0.67	863	4.61
deepsjeng (AI chess)	2216	0.57	698	2.05
Geometric mean				2.36

03

The Power Wall

Dennard scaling - CMOS power - Frequency limits

Frequency Scaling & the Rise of the Power Wall

1978 – 1989

15% / year

Early scaling: VAX → MIPS M2000
5 MHz → 25 MHz

1990 – 2004

40% / year

Dennard scaling: Alpha, Pentium
150 MHz → 3200 MHz

2005 – present

~2% / year

Power wall hits: clock rate plateaus
Intel Core i7 4200 MHz in 2017

Dennard scaling: Enabler of rapid scaling

Rule: As transistors shrink, their power density remains constant → smaller = faster **and** no extra heat.

Breakdown (~2005): Voltage could no longer be scaled proportionally, so shrinking transistors no longer kept power density flat. Clock increases now generate unsustainable heat.

CMOS Power Equation and The Power Wall

$$Power = P_{static} + P_{dynamic} = P_{static} + \frac{1}{2} C V_{DD}^2 f$$

C Capacitance

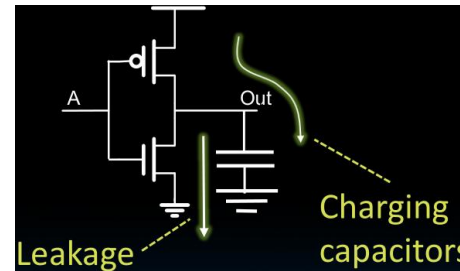
Shrinks as transistors get smaller.

V_{DD}^2 Supply Voltage (squared)

Cannot be reduced fast enough → dominant problem.

f Clock Frequency

Higher frequency = directly more power.

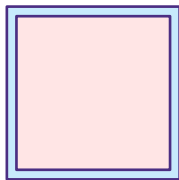


- > Raising clock rate increases the chip's power density
- > Nearly all consumed electrical power is converted into thermal energy (heat)
- > We've hit **the Power Wall** (limit of practical air-cooling); further frequency increases would overheat the chip

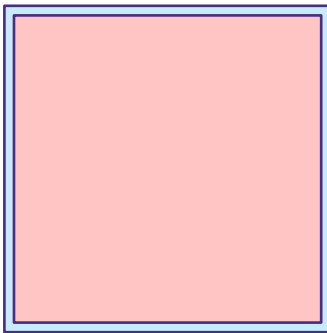
Industry response: Shift from single fast cores to multiple smaller cores.

The Architecture Pivot: Why Multicore?

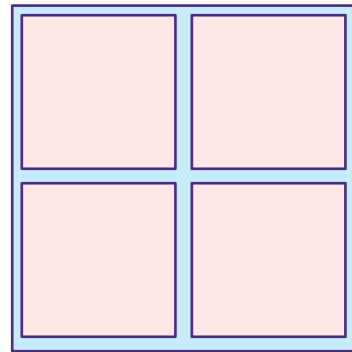
Pollack's rule: processor performance grows with square-root of die area



Perf = 1
Area = 1



Perf = 2
Area = 4



Perf = 4
Area = 4

💡 *Multicore delivers higher performance within the same power/area budget, improving performance per watt*

04

Amdahl's Law

Speedup formula - Limits of parallelism

Amdahl's Law: Limits of Any Optimization

$$\text{Speedup}(E) = \frac{1}{\frac{f}{s} + (1-f)} \quad \text{where } f = \text{fraction that can be enhanced, } s = \text{speedup factor of that fraction}$$

How the Formula Arises

> If enhancement E speeds up fraction f by factor s :

- New time = Old time $\times \left[\frac{f}{s} + (1-f) \right]$
- $\text{Speedup}(E) = \frac{\text{Old time}}{\text{New time}} = \frac{1}{\frac{f}{s} + (1-f)}$

> The speedup limit:

- As $s \rightarrow \infty$: max speedup = $\frac{1}{1-f}$

Example:

A program runs in 100s, where multiply takes 80s. How much do we have to improve the multiplication speed

Q1: If we want the program to run 4× faster?

$$0.25 = 0.8/s + 0.2 \rightarrow s = 16$$

Must speed up multiply by 16× ✓

Q2: If we want the program to run 5× faster?

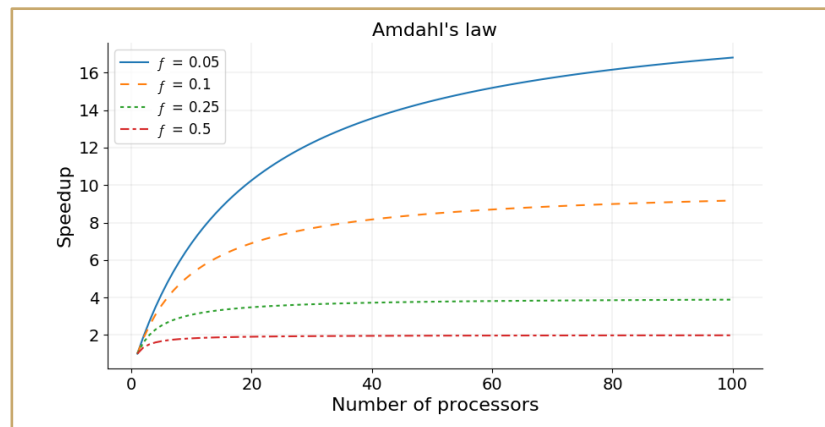
$$0.20 = 0.8/s + 0.2 \rightarrow s = \infty \rightarrow \text{Impossible!}$$

Even infinitely fast multiply can't achieve 5×

💡 The sequential fraction $(1-f)$ sets a hard limit on total speedup, even with unlimited parallelism.

Multicore & Amdahl: The Parallel Speedup Ceiling

<i>Cores</i> <i>Seq. fraction</i>	2	4	8	16	32	∞
f = 0%	2.0×	4.0×	8.0×	16.0×	32.0×	∞
f = 10%	1.8×	3.1×	4.7×	6.4×	7.8×	10.0×
f = 20%	1.7×	2.5×	3.3×	4.0×	4.4×	5.0×
f = 50%	1.3×	1.6×	1.8×	1.9×	1.9×	2.0×



Even 10% sequential code...

- > Speedup is capped at 10× **regardless** of how many cores you add
- > No amount of hardware solves a SW bottleneck

Implication for architects

- > Adding more cores has **diminishing** returns
- > Focus should shift to reducing the sequential fraction through algorithmic redesign

05

Future Trends

DSAs · Compiler Technology · Open ISA · Emerging Paradigms

The Post-Multicore Challenge

Multicore helped — but three walls still constrain performance growth:

Power Wall (revisited)

More cores = more power. Dark silicon: we can only power ~20% of a chip at full speed at once.

Memory Wall

CPU speed grows faster than DRAM bandwidth. Programs stall waiting for data, not compute.

Amdahl's Wall

Sequential fractions cap speedup regardless of core count. Adding cores yields diminishing returns.

Response: *a new wave of strategies — hardware specialization, smarter compilers, and new computing paradigms.*

Domain-Specific Architectures (DSAs)

Abandon one-size-fits-all. Build hardware tuned for a single problem class.

GPU

Graphics Processing Unit

Why: Thousands of simple cores; excellent for massively parallel float math.

Use: Deep learning training, scientific simulation, image rendering.

10–100×

vs CPU on DNN workloads

TPU

Tensor Processing Unit (Google)

Why: Systolic array of multiply-accumulate units optimised for matrix ops (INT8/BF16).

Use: Neural network inference & training at data-centre scale.

15–30×

vs GPU on large-batch inference

NPU

Neural Processing Unit (edge)

Why: Low-power inference engine in smartphones & embedded devices.

Use: On-device ML: face recognition, voice, AR — without cloud round-trip.

5–10×

vs ARM CPU at <1W

FPGA

Field-Programmable Gate Array

Why: Reconfigurable logic. Intermediate between CPU flexibility and ASIC efficiency.

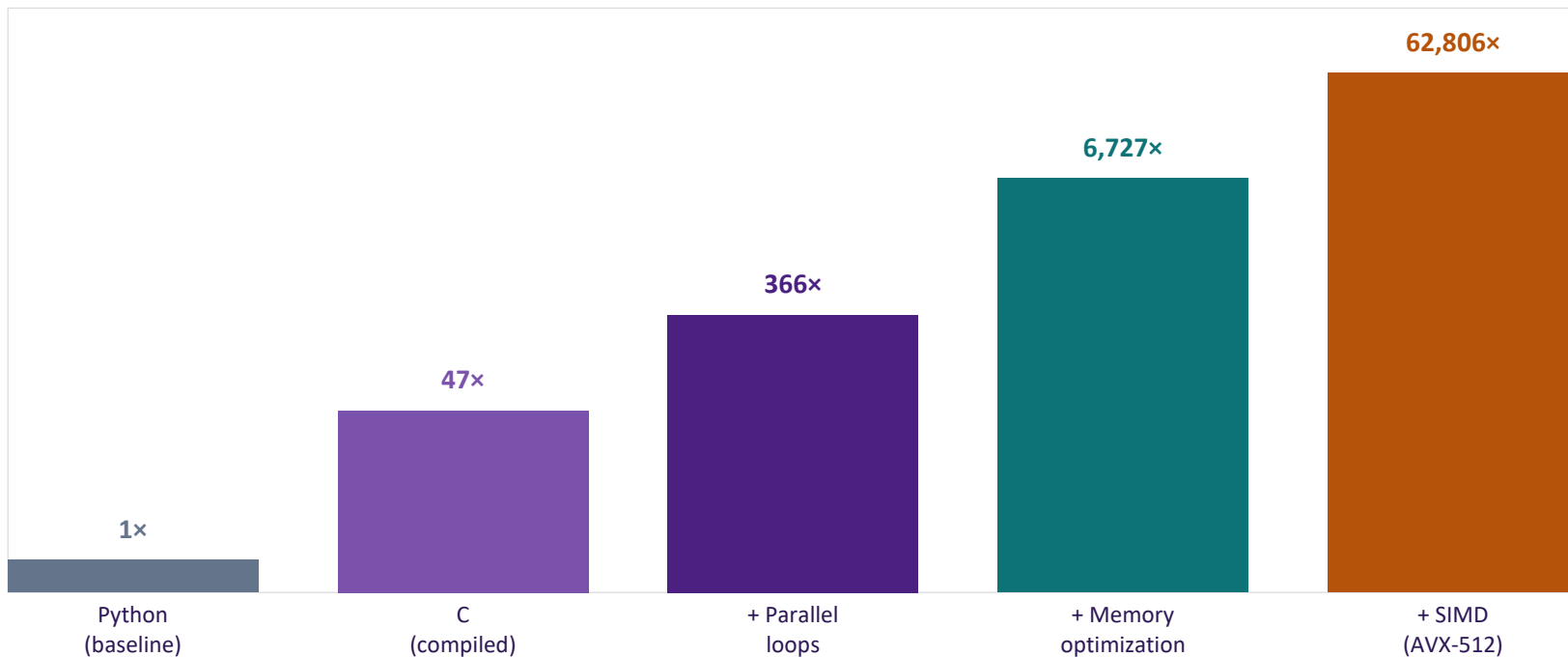
Use: Network packet processing, HFT, prototyping custom accelerators.

5–50×

app-specific vs CPU

The Full Software Stack

Matrix multiply speedup over native Python — same hardware, different layers of the stack:



Strategies for Continued Performance Growth

 Software / Compiler	 Architecture	 Emerging Paradigms
Advanced compiler optimization Auto-vectorization (SIMD), loop tiling, polyhedral scheduling squeeze 10–60× out of existing hardware.	RISC-V & Open ISA Open architectures let researchers and startups innovate without licensing barriers - accelerating DSA adoption.	Processing-in-Memory (PIM) Moving compute to where the data resides eliminates the memory wall for data-intensive operations.
Domain-Specific Languages (DSLs) Languages like CUDA, Halide, TVM let programmers express parallelism the compiler maps to the right hardware.	Chiplet / 3D stacking Chiplets (disaggregated compute and memory) + advanced packaging → lower latency and higher bandwidth.	Quantum & neuromorphic Quantum for specific combinatorial tasks; neuromorphic chips (Intel Loihi) for ultra-low-power event-driven AI.
 Key insight: <i>Performance will continue to grow — but through vertical co-design of algorithms, compilers, ISA, and custom silicon, not by simply scaling clock rate or core count.</i>		

KEY TAKEAWAYS

1

Performance is context-dependent

Always define your metric (latency vs. throughput) before comparing systems.

2

The Iron Law

CPU Time = IC × CPI × Clock Period. Reduce any factor - but watch for trade-offs between them.

3

MIPS can mislead

Higher throughput ratings don't guarantee shorter execution time. Use benchmarks for real comparisons.

4

The Power Wall stopped clock scaling

~2005: voltage could not shrink further. Response: multicore, then domain-specific architectures.

5

Amdahl's Law is a hard ceiling

Optimizing a fraction f by $s\times$ yields speedup $\leq 1/(1-f)$. Redesign the sequential part, not just the fast part.

Next Lecture: Instruction Set Architecture — P&H sections 2.1–2.7